



**PALADIN**  
BLOCKCHAIN SECURITY

# Smart Contract Security Assessment

Final Report

For Ethena

24 October 2025



[paladinsec.co](https://paladinsec.co)



[info@paladinsec.co](mailto:info@paladinsec.co)

# Table of Contents

|                                  |    |
|----------------------------------|----|
| Table of Contents                | 2  |
| Disclaimer                       | 3  |
| 1 Overview                       | 4  |
| 1.1 Summary                      | 4  |
| 1.2 Contracts Assessed           | 4  |
| 1.3 Findings Summary             | 5  |
| 1.3.1 xUSDOFTUpgradeable         | 6  |
| 1.3.2 RateLimiterUpgradeable     | 6  |
| 1.3.3 OFTOwnable2StepUpgradeable | 6  |
| 1.3.4 OFTEvents                  | 7  |
| 1.3.5 OFTErrors                  | 7  |
| 2 Findings                       | 8  |
| 2.1 xUSDOFTUpgradeable           | 8  |
| 2.1.1 Privileged Functions       | 9  |
| 2.1.2 Issues & Recommendations   | 10 |
| 2.2 RateLimiterUpgradeable       | 15 |
| 2.2.1 Issues & Recommendations   | 16 |
| 2.3 OFTOwnable2StepUpgradeable   | 22 |
| 2.3.1 Privileged Functions       | 22 |
| 2.3.2 Issues & Recommendations   | 23 |
| 2.4 OFTEvents                    | 25 |
| 2.4.1 Issues & Recommendations   | 25 |
| 2.5 OFTErrors                    | 26 |
| 2.5.1 Issues & Recommendations   | 26 |

# Disclaimer

Paladin Blockchain Security ("Paladin") has conducted an independent audit to verify the integrity of and highlight any vulnerabilities or errors, intentional or unintentional, that may be present in the codes that were provided for the scope of this audit. This audit report does not constitute agreement, acceptance or advocacy for the Project that was audited, and users relying on this audit report should not consider this as having any merit for financial advice in any shape, form or nature. The contracts audited do not account for any economic developments that may be pursued by the Project in question, and that the veracity of the findings thus presented in this report relate solely to the proficiency, competence, aptitude and discretion of our independent auditors, who make no guarantees nor assurance that the contracts are completely free of exploits, bugs, vulnerabilities or deprecation of technologies. Further, this audit report shall not be disclosed nor transmitted to any persons or parties on any objective, goal or justification without due written assent, acquiescence or approval by Paladin.

All information provided in this report does not constitute financial or investment advice, nor should it be used to signal that any persons reading this report should invest their funds without sufficient individual due diligence regardless of the findings presented in this report. Information is provided 'as is', and Paladin is under no covenant to the completeness, accuracy or solidity of the contracts audited. In no event will Paladin or its partners, employees, agents or parties related to the provision of this audit report be liable to any parties for, or lack thereof, decisions and/or actions with regards to the information provided in this audit report.

Cryptocurrencies and any technologies by extension directly or indirectly related to cryptocurrencies are highly volatile and speculative by nature. All reasonable due diligence and safeguards may yet be insufficient, and users should exercise considerable caution when participating in any shape or form in this nascent industry.

The audit report has made all reasonable attempts to provide clear and articulate recommendations to the Project team with respect to the rectification, amendment and/or revision of any highlighted issues, vulnerabilities or exploits within the contracts provided. It is the sole responsibility of the Project team to sufficiently test and perform checks, ensuring that the contracts are functioning as intended, specifically that the functions therein contained within said contracts have the desired intended effects, functionalities and outcomes of the Project team.

Paladin retains the right to re-use any and all knowledge and expertise gained during the audit process, including, but not limited to, vulnerabilities, bugs, or new attack vectors. Paladin is therefore allowed and expected to use this knowledge in subsequent audits and to inform any third party, who may or may not be our past or current clients, whose projects have similar vulnerabilities. Paladin is furthermore allowed to claim bug bounties from third-parties while doing so.

# 1 Overview

This report has been prepared for Ethena's contracts on the Ethereum network. Paladin provides a user-centred examination of the smart contracts to look for vulnerabilities, logic errors or other issues from both an internal and external perspective.

## 1.1 Summary

|                              |                                                                                                                                                                                                                           |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Project Name</b>          | Ethena                                                                                                                                                                                                                    |
| <b>URL</b>                   | <a href="https://ethena.fi/">https://ethena.fi/</a>                                                                                                                                                                       |
| <b>Platform</b>              | Ethereum                                                                                                                                                                                                                  |
| <b>Language</b>              | Solidity                                                                                                                                                                                                                  |
| <b>Preliminary Contracts</b> | <a href="https://github.com/ethena-labs/onchain-minting-internal/tree/d3fb8ab8d0fd10b8e5437c8e0898a4d74badc04b">https://github.com/ethena-labs/onchain-minting-internal/tree/d3fb8ab8d0fd10b8e5437c8e0898a4d74badc04b</a> |
| <b>Resolution 1</b>          | <a href="https://github.com/ethena-labs/onchain-minting-internal/tree/930daa2d569c57fd914a7d345c416c145ebc935d">https://github.com/ethena-labs/onchain-minting-internal/tree/930daa2d569c57fd914a7d345c416c145ebc935d</a> |

## 1.2 Contracts Assessed

| Name                       | Contract | Live Code Match |
|----------------------------|----------|-----------------|
| xUSD0FTUpgradeable         |          |                 |
| RateLimiterUpgradeable     |          |                 |
| OFTOwnable2StepUpgradeable |          |                 |
| OFTEvents                  |          |                 |
| OFTErrors                  |          |                 |

## 1.3 Findings Summary

| Severity        | Found | Resolved | Partially Resolved | Acknowledged (no change made) |
|-----------------|-------|----------|--------------------|-------------------------------|
| ● Governance    | 1     | -        | -                  | 1                             |
| ● High          | 0     | -        | -                  | -                             |
| ● Medium        | 0     | -        | -                  | -                             |
| ● Low           | 4     | 2        | -                  | 2                             |
| ● Informational | 11    | 9        | -                  | 2                             |
| Total           | 16    | 11       | -                  | 5                             |

### Classification of Issues

| Severity               | Description                                                                                                                                                                                                                                                                |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Governance</b>      | Issues under this category are where the governance or owners of the protocol have certain privileges that users need to be aware of, some of which can result in the loss of user funds if the governance's private keys are lost or if they turn malicious, for example. |
| ● <b>High</b>          | Exploits, vulnerabilities or errors that will certainly or probabilistically lead towards loss of funds, control, or impairment of the contract and its functions. Issues under this classification are recommended to be fixed with utmost urgency.                       |
| ● <b>Medium</b>        | Bugs or issues that may be subject to exploit, though their impact is somewhat limited. Issues under this classification are recommended to be fixed as soon as possible.                                                                                                  |
| ● <b>Low</b>           | Effects are minimal in isolation and do not pose a significant danger to the project or its users. Issues under this classification are recommended to be fixed nonetheless.                                                                                               |
| ● <b>Informational</b> | Consistency, syntax or style best practices. Generally pose a negligible level of risk, if any.                                                                                                                                                                            |

### 1.3.1 xUSDOFTUpgradeable

| ID | Severity | Summary                                                                                                  | Status       |
|----|----------|----------------------------------------------------------------------------------------------------------|--------------|
| 01 | GOV      | Centralized governance and upgradeability risk in OFT configuration                                      | ACKNOWLEDGED |
| 02 | LOW      | Rate limiter overcharges by counting dust in _debit (uses raw passed amount instead of amountSentLD)     | ACKNOWLEDGED |
| 03 | INFO     | Rate limits are not enforced upon receipt                                                                | ACKNOWLEDGED |
| 04 | INFO     | Rescue recipient can be blacklisted, causing destination mint to revert if recipient is also blacklisted | ✓ RESOLVED   |
| 05 | INFO     | Public burns allow arbitrary supply reduction, conflicting with the current architecture and economics   | ACKNOWLEDGED |
| 06 | INFO     | Typographical issues                                                                                     | ✓ RESOLVED   |

### 1.3.2 RateLimiterUpgradeable

| ID | Severity | Summary                                                                                                   | Status       |
|----|----------|-----------------------------------------------------------------------------------------------------------|--------------|
| 07 | LOW      | Possible zero-amount sends refresh rate limiter timestamp which could lead to temporal DoS                | ✓ RESOLVED   |
| 08 | LOW      | Rate limit updates ignore prior decay, causing over or under-conservative capacity after parameter change | ACKNOWLEDGED |
| 09 | LOW      | Incorrect ERC-7201 storage slot                                                                           | ✓ RESOLVED   |
| 10 | INFO     | _amountCanBeSent will revert if the limit is set is set excessively high, e.g. type(uint256).max          | ✓ RESOLVED   |
| 11 | INFO     | Typographical issues                                                                                      | ✓ RESOLVED   |

### 1.3.3 OFTOwnable2StepUpgradeable

| ID | Severity | Summary                                                             | Status     |
|----|----------|---------------------------------------------------------------------|------------|
| 12 | INFO     | Inconsistent _disableInitializers invoke between abstract contracts | ✓ RESOLVED |
| 13 | INFO     | address(0) can claim ownership as per formal verification           | ✓ RESOLVED |
| 14 | INFO     | Typographical issues                                                | ✓ RESOLVED |

### 1.3.4 OFTEvents

| ID | Severity | Summary              | Status     |
|----|----------|----------------------|------------|
| 15 | INFO     | Typographical issues | ✓ RESOLVED |

### 1.3.5 OFTErrors

| ID | Severity | Summary              | Status     |
|----|----------|----------------------|------------|
| 16 | INFO     | Typographical issues | ✓ RESOLVED |

## 2 Findings

---

### 2.1 xUSDOFTUpgradeable

xUSDOFTUpgradeable is the canonical, upgradeable ERC20 token that implements LayerZero's Omnichain Fungible Token (OFT). During a cross-chain send, the token is burned on the source chain and minted on the destination chain; amounts are normalized across chains using LayerZero's shared-decimals model with dust removal to ensure values are representable everywhere. The contract exposes standard ERC20 behavior (including ERC20Permit for gasless approvals) while adding cross-chain messaging through the OFT base. The contract has some pre-defined roles - minters, blacklisters and an owner.

Minting is restricted to approved minters chosen by the owner. Burning follows ERC20Burnable rules: holders can burn their own tokens and approved spenders can burn via allowance. Blacklist checks apply to all movements (local and cross-chain). If a cross-chain transfer targets a blacklisted recipient, the mint is redirected to the rescue recipient.

A built-in, per-destination rate limiter governs outbound cross-chain flow. For each destination endpoint ID, the contract tracks an "amount in flight" that linearly decays over a configurable time window. Before sending, the contract checks remaining capacity and updates the in-flight amount and timestamp; unconfigured destinations have zero capacity by default, so administrators must explicitly enable new routes.



## 2.1.1 Privileged Functions

- `addMinter`
- `removeMinter`
- `addBlacklister`
- `removeBlacklister`
- `addToBlacklist`
- `removeFromBlacklist`
- `mint`
- `setRescueRecipient`
- `rescueBlacklistedTokens`
- `setRateLimits`
- `_authorizeUpgrade`

## 2.1.2 Issues & Recommendations

|                |                                                                                                                                                                                                                                                                                                                                                                         |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Issue #01      | Centralized governance and upgradeability risk in OFT configuration                                                                                                                                                                                                                                                                                                     |
| Severity       | <div><div></div>GOVERNANCE</div>                                                                                                                                                                                                                                                                                                                                        |
| Description    | <p>The contract owner has full control over critical protocol functions such as selecting minters, managing blacklists, rescuing assets from blacklisted users, and upgrading the contract logic. This centralization means protocol behavior and user balances can be changed at any time by the owner, including adding/removing roles or upgrading to new logic.</p> |
| Recommendation | <ul style="list-style-type: none"><li>- Use a well-secured multisig wallet as the contract owner.</li><li>- Require multiple independent signers for all sensitive actions.</li><li>- Document the governance setup and emergency procedures.</li></ul>                                                                                                                 |
| Resolution     | <div><div></div>ACKNOWLEDGED</div> <p>"Ethena protocol is secured by 5/11 multisigs on all chains with multiple independent signers and a hardened signing process. Although this is not a constraint that can be enforced for third party users of these contracts we generally feel that private key security falls outside of the scope of the audit."</p>           |

**Issue #02****Rate limiter overcharges by counting dust in \_debit (uses raw passed amount instead of amountSentLD)****Severity** LOW SEVERITY**Description**

In the OFT send path, \_debit updates the rate limiter with the raw amount that was passed; however, the OFT core removes dust via \_removeDust within \_debitView/\_debit, so the actual amount sent can be lower than passed.

This mismatch causes the rate limiter to consume more capacity than actually sent, leading to premature RateLimitExceeded reverts.

**Recommendation**

Consume rate limit on the exact amount the OFT will send by deriving amounts with \_debitView and use amountSentLD for the limiter before burning.

```
function _debit(address _from, uint256 _amountLD, uint256
_minAmountLD, uint32 _dstEid)
    internal
    virtual
    override
    returns (uint256 amountSentLD, uint256
amountReceivedLD)
{
    // Quote the exact amounts that will be sent/received
(dedusted)
    (uint256 sentLD, /*uint256 receivedLD*/) =
_debitView(_amountLD, _minAmountLD, _dstEid);

    // Consume rate limit using the actual amount that will
be debited
    _checkAndUpdateRateLimit(_dstEid, sentLD);

    return super._debit(_from, _amountLD, _minAmountLD,
_dstEid);
}
```

**Resolution** ACKNOWLEDGED

## Severity

 INFORMATIONAL

## Description

If a chain malfunctions and starts to inaccurately report its status, the code can still bypass the limit on that chain and send as many tokens to honest chains as desired. The rate limits will not prevent this.

This is nuanced by the fact that we always want receipt to eventually deliver, and receipt rate limits can reduce that guarantee.

## Recommendation

Consider integrating with features such as precrime or adding your own validators to ensure that the state on all supported chains is accurate and honest.

## Resolution

 ACKNOWLEDGED

"Docs added. A chain malfunctioning to the degree that it starts to inaccurately report its status such that the rate limit can be bypassed is truly an edge case event. Even adding a rate limit check on the destination chain does not completely remove this risk although it does reduce the risk surface area. In a scenario where we do have a critical chain issue which causes the contracts to misbehave, the intended remedy is to remove the wiring for the chain such that there is no valid destination. After that a cleanup process can commence where tokens can be recovered by the protocol if necessary."

|                       |                                                                                                                                                                                                                                                                                                                                                |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Issue #04</b>      | <b>Rescue recipient can be blacklisted, causing destination mint to revert if recipient is also blacklisted</b>                                                                                                                                                                                                                                |
| <b>Severity</b>       |  INFORMATIONAL                                                                                                                                                                                                                                                |
| <b>Description</b>    | In xUSD0FTUpgradeable, _credit redirects minting to the configured rescue recipient when the destination recipient is blacklisted. If rescueRecipient itself is blacklisted, the overridden _update check reverts with OFTErrors.BlacklistException, causing destination execution failures. This can stall packets until manual intervention. |
| <b>Recommendation</b> | Disallow a blacklisted address to be set as rescueRecipient, and disallow the current rescueRecipient to be added to the blacklist.                                                                                                                                                                                                            |
| <b>Resolution</b>     |  RESOLVED                                                                                                                                                                                                                                                     |

|                       |                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Issue #05</b>      | <b>Public burns allow arbitrary supply reduction, conflicting with the current architecture and economics</b>                                                                                                                                                                                                                                                                                                |
| <b>Severity</b>       |  INFORMATIONAL                                                                                                                                                                                                                                                                                                            |
| <b>Description</b>    | Any holder can burn their own xUSD0FT and any approved spender can burn someone else's xUSD0FT via allowance (through ERC20Burnable functionality). This causes supply to shrink at any time outside the protocol's controlled mint or redeem flows, which contradicts the documentation's intent that supply changes are protocol-managed.                                                                  |
| <b>Recommendation</b> | Restrict burn operations so that only allowed minters can invoke it.                                                                                                                                                                                                                                                                                                                                         |
| <b>Resolution</b>     |  ACKNOWLEDGED<br><p>"The docs have been modified to make this clearer. We are of the opinion that since the burn and burnFrom functions are public in the default implementation and there is no economic, technical or operational risk to users burning their tokens that this functionality should remain as is.."</p> |

## Severity

 INFORMATIONAL

## Description

The natspec of `addToBlacklist` and `removeFromBlacklist` incorrectly suggests the parameter should be a single address:

```
* @notice Adds an address to the blacklist
* @dev Only callable by a blacklister
* @param _blacklist Address to blacklist
* @dev Emits BlacklistAdded event on success
* @dev Reverts if address is zero or already blacklisted
* @dev Blacklisted addresses cannot send or receive tokens

* @notice Removes an address from the blacklist
* @dev Only callable by a blacklister
* @param _blacklist Address to remove from blacklist
* @dev Emits BlacklistRemoved event on success
* @dev Reverts if address is not blacklisted
* @dev Removed addresses can resume normal token operations
```

—

The natspec above `_credit` incorrectly states that `_srcEid` is the address of the destination chain when it is actually the origin chain for the bridge transaction:

```
function _credit(address _to, uint256 _amountLD, uint32
_srcEid)
```

—

`xUSD0FTUpgradeable` contract imports `0FTUpgradeable` but does not reference it directly. `OFT` integration comes via `0FTOwnable2StepUpgradeable`, making the import redundant. Consider removing the unused import.

## Recommendation

Consider fixing the above issues.

## Resolution

 RESOLVED

---

## 2.2 RateLimiterUpgradeable

RateLimiterUpgradeable is an abstract, upgradeable helper that adds per-destination throughput controls to cross-chain token transfers.

For each LayerZero destination, it stores a maximum amount (limit) and a time window (window), tracks how much is currently “in flight” and applies linear decay over time so capacity gradually frees up - when checking capacity, the contract calculates how much has decayed since the last update and subtracts that from the in flight before determining if the new amount can be accommodated, which means that during one window period a more than the actual limit amount can be sent since this simulate a leaking bucket.



## 2.2.1 Issues & Recommendations

### Issue #07

Possible zero-amount sends refresh rate limiter timestamp which could lead to temporal DoS

#### Severity

LOW SEVERITY

#### Description

In the rate limiter, `_checkAndUpdateRateLimit` updates last updated time even when the amount sent is 0. If the initial configuration has a high limit which was reached and then the limit is decreased by a large amount, users can spam zero-amount sends to continuously refresh `lastUpdated`, preventing appropriate reset of `amountInFlight` and keeping the path blocked longer than intended.

Additionally, if the new limit is lower and the previous limit was reached, the existing `amountInFlight` may exceed the new limit, causing the new window to be closed until enough time passes for decay to bring `amountInFlight` below the new limit.

#### Recommendation

Ignore zero amounts in `_checkAndUpdateRateLimit` so they do not touch `lastUpdated` or `amountInFlight`. Optionally, also skip the limiter update in the OFT debit path when the dedusted `amountSentLD == 0`.

Consider setting `amountInFlight` to the maximum limit during an update if it exceeds the limit to avoid long decay periods.



---

```
function _checkAndUpdateRateLimit(uint32 _dstEid, uint256
_amount) internal {
+   // Ignore zero-amount updates to prevent refreshing the
decay window via spam
+   if (_amount == 0) return;
    RateLimiterUpgradeableStorage storage $ =
_getRateLimiterUpgradeableStorage();
    // @dev by default dstEid that have not been explicitly
set will return amountCanBeSent == 0
    RateLimit storage rl = $.rateLimits[_dstEid];

    (uint256 currentAmountInFlight, uint256 amountCanBeSent)
= _amountCanBeSent(rl.amountInFlight, rl.lastUpdated,
rl.limit, rl.window);
    if (_amount > amountCanBeSent) revert
RateLimitExceeded();

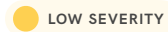
    // update the storage to contain the new amount and
current timestamp
    rl.amountInFlight = currentAmountInFlight + _amount;
    rl.lastUpdated = block.timestamp;
}
```

---

## Resolution



## Severity



## Description

When updating rate limits, `_setRateLimits` writes limit and window without first checkpointing decay using the prior parameters. This retroactively applies the new rate/window to the old in-flight amount, freeing less or more capacity than elapsed time under the old configuration would imply (e.g., 30 min into 100/h should free 50, but after lowering to 50/h it frees only 25).

## Recommendation

Before setting new limit/window, checkpoint using the old limit/window to persist elapsed decay:

- Compute `(currentAmountInFlight, _) = _amountCanBeSent(r1.amountInFlight, r1.lastUpdated, r1.limit, r1.window)`
- Set `r1.amountInFlight = currentAmountInFlight` and `r1.lastUpdated = block.timestamp`
- Set the new limit/window



---

```

function _setRateLimits(RateLimitConfig[] memory
_rateLimitConfigs) internal {
    RateLimiterUpgradeableStorage storage $ =
_getRateLimiterUpgradeableStorage();
    unchecked {
        for (uint256 i = 0; i < _rateLimitConfigs.length; i+
+) {
            RateLimit storage rl =
$.rateLimits[_rateLimitConfigs[i].dstEid];
            if (_rateLimitConfigs[i].window == 0) revert
OFTErrors.InvalidWindow();

            // Checkpoint decay using the OLD config so we don't
retroactively apply the new rate/window
            (uint256 currentAmountInFlight, ) =
                _amountCanBeSent(rl.amountInFlight,
rl.lastUpdated, rl.limit, rl.window);
            rl.amountInFlight = currentAmountInFlight;
            rl.lastUpdated = block.timestamp;

            // Now apply the new config
            rl.limit = _rateLimitConfigs[i].limit;
            rl.window = _rateLimitConfigs[i].window;
        }
    }
    emit RateLimitsChanged(_rateLimitConfigs);
}

```

---

## Resolution



| Issue #09 Incorrect ERC-7201 storage slot |                                                                                                                                                                                                     |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Severity                                  |  LOW SEVERITY                                                                                                      |
| Description                               | The storage slot is incorrect:<br><code>bytes32 private constant<br/> RateLimiterUpgradeableStorageLocation =<br/> 0x8327c26dd5e5ceb4f0d1337dcb7f7536980a6eda04f5f7fb6577c3c861<br/> cded00;</code> |
| Recommendation                            | Change it to the correct one:<br><code>0xf40fb8ff74ec76132acbf7c7d3a15179466240c00e91ec9e6b83495e3a3<br/> e51600</code>                                                                             |
| Resolution                                |  RESOLVED                                                                                                          |

| Issue #10 <code>_amountCanBeSent</code> will revert if the limit is set is set excessively high, e.g. <code>type(uint256).max</code> |                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Severity                                                                                                                             |  INFORMATIONAL                                                                                                                                                                                                                                                                                                                                            |
| Description                                                                                                                          | If limit is set to <code>uint256.max</code> , the rate limit check will revert after any time has elapsed since <code>lastUpdated</code> , because <code>_amountCanBeSent</code> computes <code>(_limit * timeSinceLastDeposit) / _window</code> , which overflows in Solidity 0.8+ when <code>_limit</code> is <code>uint256.max</code>                                                                                                     |
| Recommendation                                                                                                                       | There are several options to resolve this. A maximum limit can be considered to prevent this scenario. Alternatively, a special value can be used for disabling the rate limiter. A non-reverting mul-div implementation could be used, though these are quite complex so we do not recommend it. Finally, the multiplication can simply be capped to the maximum <code>uint</code> if it overflows, which is the most recommended solution. |
| Resolution                                                                                                                           |  RESOLVED                                                                                                                                                                                                                                                                                                                                                 |

| Issue #11      | Typographical issues                                                                                                                                                                                                                                                                                               |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Severity       |  INFORMATIONAL                                                                                                                                                                                                                    |
| Description    | <p>The comment in the following struct incorrectly states that it stores a mapping of addresses to rate limit statuses:</p> <pre>struct RateLimiterUpgradeableStorage {<br/>    /// @notice Mapping of addresses to rate limit status<br/>    mapping(uint32 dstEid =&gt; RateLimit limit) rateLimits;<br/>}</pre> |
| Recommendation | Consider fixing the issue.                                                                                                                                                                                                                                                                                         |
| Resolution     |  RESOLVED                                                                                                                                                                                                                         |



---

## 2.3 OFTOwnable2StepUpgradeable

OFTOwnable2StepUpgradeable is an abstract base that combines LayerZero's upgradeable OFT token with a safe two-step ownership transfer. This means that changes happen in two stages — the current owner invokes `transferOwnership` to set a pending owner, and only that pending owner can finalize by accepting it.

Renouncing ownership is explicitly disabled to preserve required admin capabilities. This contract stores the pending owner using ERC-7201 namespaced storage pattern following OpenZeppelin `Ownable2StepUpgradeable` implementation.

### 2.3.1 Privileged Functions

- `transferOwnership`
- `acceptOwnership`
- `renounceOwnership`



## 2.3.2 Issues & Recommendations

|                       |                                                                                                                                                                                                                                                                                 |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Issue #12</b>      | <b>Inconsistent <code>_disableInitializers</code> invoke between abstract contracts</b>                                                                                                                                                                                         |
| <b>Severity</b>       |  INFORMATIONAL                                                                                                                                                                                 |
| <b>Description</b>    | The <code>OFTOwnable2StepUpgradeable</code> and <code>RateLimiterUpgradeable</code> are both upgradable abstract contracts, but while <code>OFTOwnable2StepUpgradeable</code> invokes <code>_disableInitializers</code> in its constructor, the rate limiter contract does not. |
| <b>Recommendation</b> | Make the appropriate changes so that they are consistent                                                                                                                                                                                                                        |
| <b>Resolution</b>     |  RESOLVED                                                                                                                                                                                      |

|                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Issue #13</b>      | <b><code>address(0)</code> can claim ownership as per formal verification</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Severity</b>       |  INFORMATIONAL                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Description</b>    | <p>Formal verification of ownership transfer rules reveals that <code>address(0)</code> can technically claim ownership if it is ever set as <code>_pendingOwner</code>. The rule "Nobody can claim ownership when an owner is not proposed" would fail, because the system allows <code>address(0)</code> to be treated as a valid pending owner.</p> <p>Although this does not constitute a practical exploit since the zero address is highly unlikely to be the transaction sender and most EVMs block transactions from it, it remains a formal edge case worth documenting.</p> |
| <b>Recommendation</b> | For completeness in formal specifications, consider adding an explicit check:<br><pre>require(newOwner != address(0), "New owner cannot be zero address");</pre>                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Resolution</b>     |  RESOLVED                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

| Issue #14      Typographical issues |                                                                                                                                                                                                                                                                              |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Severity                            |  INFORMATIONAL                                                                                                                                                                              |
| Description                         | <p>The comment in the following struct incorrectly states that it stores a mapping of addresses to minter statuses:</p> <pre>struct OFTOwnable2StepUpgradeableStorage {<br/>    /// @notice Mapping of addresses to minter status<br/>    address _pendingOwner;<br/>}</pre> |
| Recommendation                      | Consider fixing the issue.                                                                                                                                                                                                                                                   |
| Resolution                          |  RESOLVED                                                                                                                                                                                   |





## 2.4 OFTEvents

OFTEvents is an abstract base that combines LayerZero's upgradeable OFT token with a safe two-step ownership transfer. This means that changes happen in two stages — the current owner invokes `transferOwnership` to set a pending owner, and only that pending owner can finalize by accepting it.

Renouncing ownership is explicitly disabled to preserve required admin capabilities. This contract stores the pending owner using ERC-7201 namespaced storage pattern following OpenZeppelin `Ownable2StepUpgradeable` implementation.

### 2.4.1 Issues & Recommendations

| Issue #15      | Typographical issues                                                                                                                                                                                                                                                                                                                                                                   |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Severity       |  INFORMATIONAL                                                                                                                                                                                                                                                                                      |
| Description    | <p>The parameter name in the event incorrectly states that the caller would be a minter, when in fact it would be just the user.</p> <ul style="list-style-type: none"><li>* @param amount Amount of tokens burned</li><li>* @param minter Address of the user who performed the burn event</li></ul> <pre>Burned(address indexed from, uint256 amount, address indexed minter);</pre> |
| Recommendation | Consider fixing the issue.                                                                                                                                                                                                                                                                                                                                                             |
| Resolution     |  RESOLVED                                                                                                                                                                                                                                                                                           |

## 2.5 OFTErrors

OFTErrors is a centralized custom errors library used in the xUSD OFT ecosystem. It defines all reusable error types.

### 2.5.1 Issues & Recommendations

| Issue #16      |                                                                                                                                                                                       | Typographical issues |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| Severity       |  INFORMATIONAL                                                                                       |                      |
| Description    | The @notice above OFTErrors.BlacklistException incorrectly states it is "Thrown when trying to set rate limits by a non-owner". In practice, this error signals blacklist violations. |                      |
| Recommendation | Update the natspec to accurately reflect usage, e.g., "Thrown when a blacklist rule is violated (sender, from, or to is blacklisted)".                                                |                      |
| Resolution     |  RESOLVED                                                                                          |                      |



**PALADIN**  
BLOCKCHAIN SECURITY